

# SpotCC: Facilitating Coded Computation for Prediction Serving Systems on Spot Instances

Lin Wang, Yuchong Hu<sup>✉</sup>, Ziling Duan, Mingqi Li, Chenxuan Yao, Feifan Liu,  
Xiaolu Li, Leihua Qin, and Dan Feng

Huazhong University of Science and Technology

{wanglin2021, yuchonghu, duanziling, mingqili1006, deadfffool, ffliu, lixl666, lhqin, dfeng}@hust.edu.cn

**Abstract**—The growing adoption of prediction serving systems (PSSes) has made cost-saving deployment on preemptible spot instances crucial, yet frequent preemptions severely harm availability. While coded computation (CC) can keep availability cost-effectively by encoding original jobs into parity ones, its direct application to spot instances incurs prohibitive decoding overhead and tail latency under frequent preemptions. We identify two findings for optimization: (i) decoding asymmetry (only original job failures require decoding); (ii) preemption unevenness (variation in preemption rates across cloud regions).

Leveraging these findings, we propose SpotCC, a new CC framework that strategically dispatches parity jobs to high-preemption (volatile) regions and original jobs to low-preemption (stable) regions. SpotCC designs locality-based and fine-grained volatility identification to reduce decoding operations and mitigate job congestion, respectively, and adaptively tunes configurations for decoding minimization. Experiments show that SpotCC improves P99 latency by 83.9% over state-of-the-arts, while maintaining ultra-low monetary costs.

## I. INTRODUCTION

With the surge of AI services, the *prediction serving system* (PSS for short), which hosts deployed models for inference and returns prediction results to users, has been widely adopted in both academia [19], [34], [54], [59], [21] and industry [3], [6], [5]. As PSSes typically operate in clouds and require expensive GPU accelerators for serving AI models [38], their serving cost has become a significant concern (often up to  $10\times$  higher than traditional CPU-based systems [40]). Moreover, real-world AI workloads exhibit frequent and unpredictable traffic spikes [38], forcing providers to over-provision resources [40], [34], [64] (i.e., allocate more than needed for average traffic to handle peak loads), thus further exacerbating the serving cost. To reduce this cost, many recent studies [41], [40] explore deploying models on cheap *preemptible instances* (e.g., AWS spot instances [13]), a class of cloud resources that can be up to 90% cheaper than *on-demand* instances [41] but can be preempted at any time. While the preemptible instances offer significant cost savings, their frequent preemption inherently introduces substantial unavailability [55], [20], [40].

A typical way for PSSes to address unavailability caused by slowdowns or failures—referred to as *stragglers*—is to apply *replication* [34], [18], which replicates original prediction jobs (e.g.,  $A$  and  $B$  in Figure 1) into multiple instances as identical jobs (e.g.,  $A'$  and  $B'$  in Figure 1(a)), such that, even if any instance becomes a straggler during inference, users can still obtain a timely prediction result from the remaining replicas.

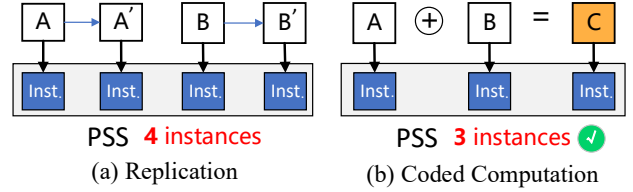


Fig. 1: Benefit of coded computation over replication in PSSes.

Nevertheless, replication incurs substantial resource overhead, since it requires multiple redundant computations for each prediction job [34], [54], [59].

Alternatively, erasure coding techniques [49] (traditionally used in storage systems [30], [23]) have been recently adopted in PSSes, a paradigm termed *coded computation* (CC) [36], to address straggler problems with lower resource overhead than replication [31], [34], [35], [54]. As illustrated in Figure 1(b), CC encodes the original jobs  $A$  and  $B$  into an additional parity job  $C$ , where any two of the results from  $A$ ,  $B$ , and  $C$  can *decode* the results of  $A$  and  $B$  regardless of which instance becomes a straggler. In this case, tolerating one straggler requires three instances for CC versus four for replication, achieving a 25% overhead reduction.

However, when a PSS deploys CC on cheap preemptible instances for ultra-low cost, we observe that the high occurrence of preemption (e.g., 33%-55% of instances of one cluster are often preempted within one hour [55], [20]) triggers numerous slow CC decoding operations, thus significantly increasing the overall tail latency (see §III-A for details). So far, existing studies on CC for PSSes [34], [54], [59] have not explored decoding efficiency, which motivates us to be the first to address the CC decoding problem for PSSes deployed on preemptible instances.

To mitigate CC decoding overhead during frequent preemptions, we exploit two findings (see §III-B for details): i) *Decoding Asymmetry*—we find that only the failure of original jobs requires decoding, whereas parity job failures do not, which differs fundamentally from the traditional decoding paradigm in erasure-coded storage; and ii) *Preemption Unevenness*—we find that the preemption ratio (i.e., the ratio of instances preempted to all instances in a period) varies significantly across different cloud regions [40], [62], [27].

Our main idea is to proactively align decoding asymmetry with preemption unevenness to reduce preemption-induced decoding operations. Specifically, we dispatch parity jobs to

regions with high preemption ratios (*volatile regions*) and original jobs to regions with low preemption ratios (*stable regions*). In this way, volatile regions avoid decoding when preemptions occur (since parity jobs are disposable), while stable regions rarely require decoding due to their low preemption ratios.

In this paper, we propose **SpotCC**, an efficient CC mechanism for PSSes on spot instances. **SpotCC** comprises three key components: (i) a locality-based volatility identification scheme (**SpotCC-L**) to largely reduce CC decoding operations by effectively identifying volatile regions via intra-region preemption locality; (ii) a fine-grained volatility identification scheme (**SpotCC-F**) to mitigate the original job congestion problem by exploring inter-zone volatility correlation; (iii) an adaptive profiling-guided timeout scheme (**SpotCC-A**) to minimize CC decoding operations via learning-based tuning configurations. Our contributions are as follows:

- We conduct measurement analysis to highlight the high-cost challenges of deploying CC in PSSes on spot instances. We propose **SpotCC**, the first framework to address the CC decoding problem on frequent preemptions, based on two key optimization findings (§III).
- We design **SpotCC**, which first quickly identifies volatile regions via leveraging the intra-region preemption locality and then enhances the identification in a fine-grained manner via inter-zone volatility correlation. We also design an adaptive configuration tuning algorithm to minimize the CC decoding operations (§IV).
- We implement **SpotCC** atop NVIDIA Triton Inference Server [44], and our experiments via Alibaba Cloud [1] show that **SpotCC** can outperform state-of-the-arts by up to 83.9% in P99 latency (§V). **SpotCC** is open-sourced at <https://github.com/YuchongHu/SpotCC>.

## II. BACKGROUND

### A. Prediction Serving Systems on Spot Instances

**Prediction serving systems.** Prediction serving systems (PSSes) are designed to serve prediction services, one of the most fundamental and widely adopted forms of AI inference [34], [38], [59]. A PSS hosts deployed models, receives prediction jobs from users, performs inference, and returns the results to users. To support large-scale workloads, a PSS often adopts a distributed architecture. As shown in Figure 2, a typical distributed PSS consists of a frontend and multiple backend servers. The frontend dispatches jobs to deployed models on backend servers, which then perform inference and return results.

Nevertheless, distributed PSSes are susceptible to *stragglers* (slowdowns or failures) that can prolong tail latency. Thus, PSSes often employ *replication* [34], [18], which replicates prediction jobs to multiple workers and returns the first completed result. As depicted in Figure 2, jobs *A* and *B* are replicated on two servers; the prediction results  $A_r$  and  $B_r$  are then collected from the fastest-responding server. In practice, due to unpredictable traffic spikes (up to  $50\times$  the average [38]),

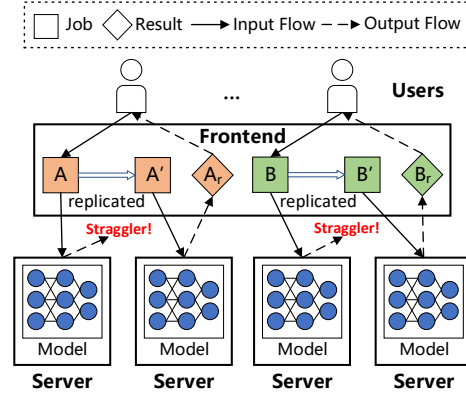


Fig. 2: Architecture of a PSS.

TABLE I: Cost of spot instances, in percentage of on-demand instances. Price obtained on July 7th, 2025.

|             | A100 | A10 | V100 | T4  | P100 | K80 |
|-------------|------|-----|------|-----|------|-----|
| AWS [4]     | 14%  | 34% | 16%  | 36% | —    | 32% |
| GCP [24]    | 46%  | —   | 39%  | 40% | 40%  | —   |
| Alibaba [1] | —    | 35% | 35%  | 24% | 31%  | —   |
| Azure [42]  | 31%  | 17% | 17%  | 30% | 19%  | 15% |

PSSes proactively *over-provision* resources [34], [54] (i.e., allocate more deployed model replicas than needed for average load) to meet peak demand.

**Spot-Instance AI serving.** AI serving systems on clouds often require many GPU instances to host AI models [38], making the serving cost much higher than that of traditional CPU-based computation systems [40]. Meanwhile, *preemptible instances* (or called *spot instances* in AWS [13]), which can be preempted by the provider at any time, have long been provided by clouds as a cost-saving option, with costs ranging from 14%-46% of on-demand instances (see Table I). Thus, to significantly reduce the serving cost, many recent studies [41], [40], [55], [20] focus on serving AI models on cheap spot instances.

We further analyze the economic model of AI serving on spot instances. A typical model comprises four cost components [46]: (i) *computation cost*  $C_{\text{compute}}$  (dominated by GPU expenses [40]), (ii) *storage cost*  $C_{\text{storage}}$  (e.g., SSD, HDD, or S3 [10]), (iii) *network cost*  $C_{\text{network}}$ , and (iv) *operational cost*  $C_{\text{operation}}$  (e.g., load balancers [7] and monitoring services [8]). The total cost is:

$$C_{\text{total}} = C_{\text{compute}} + C_{\text{storage}} + C_{\text{network}} + C_{\text{operation}} \quad (1)$$

Prior studies [40], [46], [65] and industry reports [56] consistently identify GPU computation as the dominant factor, accounting for 70-80% [56] of total expenditure. Consequently, reducing  $C_{\text{compute}}$  yields the greatest economic benefit.

Therefore, the primary rationale for using spot instances (instead of on-demand ones) for GPUs is to significantly reduce  $C_{\text{compute}}$ , thereby lowering the total cost of AI serving. For example, a GPT-3 inference service requires 128 A100 GPUs [17]. Using on-demand instances (at \$359k per GPU/year [9]) results in an annual cost of \$4.6 million. By

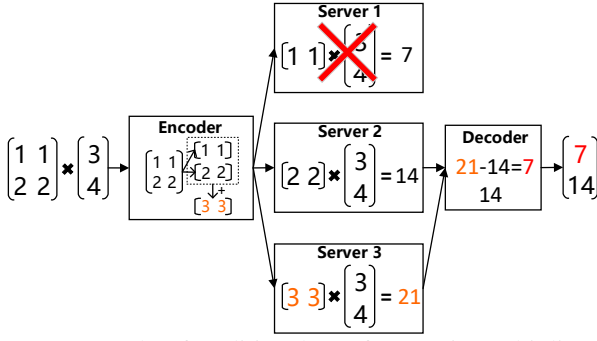


Fig. 3: Example of traditional CC for matrix multiplication.

switching to spot instances (at \$100k per GPU/year [9]), over \$3.3 million can be saved annually, yielding substantial economic benefits.

Despite their cost benefits, spot instances suffer from frequent GPU preemptions and low spot availability. Specifically, as reported by AWS Spot Instance Advisor [11], the preemption probability of spot GPU instances ( $> 20\%$ ) is far higher than that of spot CPU instances ( $< 5\%$ ). Furthermore, real-world traces [40] indicate that the availability of spot GPU instances (16.7%–90.4%) is significantly more volatile than that of spot CPU instances (95.6%–99.9%) [37]. This low availability significantly increases the risk of violating customers’ service level objectives (SLOs). Thus, many recent spot-instance-based systems on AI services [64], [41], [40] aim for meeting customers’ SLOs by proposing different strategies (e.g., latency prediction [64], optimized instance migration [41], and multi-region instance replicas [40]).

In this paper, we target spot-instance PSSes to achieve low cost and propose optimization for ensuring SLOs.

**Grace period.** Upon preemption, most cloud providers offer a *grace period* (e.g., 2 minutes for AWS [12]; 30 seconds for Azure [45]), which allows instance migration before graceful termination. The rationale behind the grace period values is to (i) provide users sufficient time to save their computation state [51], and meanwhile (ii) enable cloud providers to promptly reallocate capacity for higher-bidding customers. Therefore, the grace period value is fundamental to user service performance and providers’ ability to meet higher-bidding users’ demand. It is often carefully chosen and fixed [41], [12] since: (i) extending it would slow down providers’ resource reallocation, risking delays in serving higher-bidding customers [55], and (ii) shortening it would leave users insufficient time to preserve the computation state, leading to re-computation and degraded performance.

However, many studies [40], [29], [39], [41], [64] indicate that instance migration—including locating available instances, provisioning, and loading models—typically takes several minutes. This often exceeds the grace period, leading to failed preemption handling and subsequent AI service unavailability. Furthermore, extending the grace period would impair the cloud provider’s ability to rapidly reallocate computing resources, risking delays in fulfilling requests for higher-bidding customers [55]. To address this, recent work [40], [55]

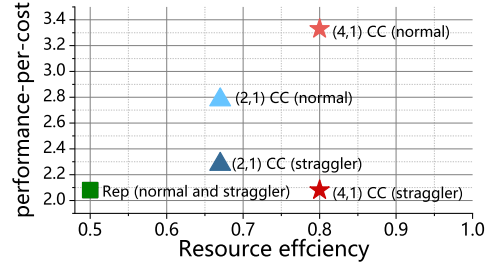


Fig. 4: Replication versus CC in distributed computing.

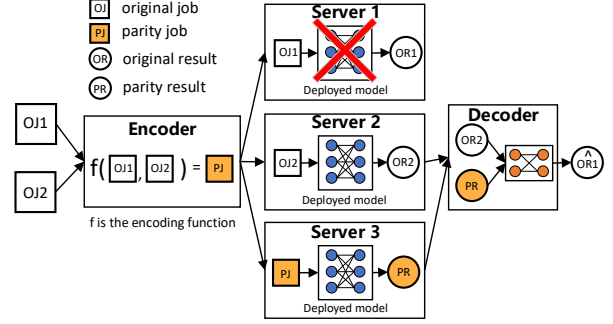


Fig. 5: Example of neural-based CC for PSSes.

has explored using instance replicas in AI model serving to guarantee high availability, yet such replica-based redundancy introduces significant resource overhead [20].

In this paper, instead of replication, we focus on leveraging another low-resource-overhead redundancy approach (§II-B) to handle preemptions for spot-instance PSSes.

## B. Coded Computation

**Erasure coding.** Erasure Coding (EC) is widely used in distributed storage systems as a low-overhead (i.e., storage-efficient) alternative to replication. A  $(k, m)$  EC encodes  $k$  original *data units* into  $m$  redundant *parity units*, ensuring that any  $k$  out of the  $k + m$  units can reconstruct the original data. The *storage efficiency* of such codes [52] is  $\frac{k}{k+m}$ . For instance, a  $(2, 1)$  EC encodes two data units  $D_1$  and  $D_2$  into a parity unit  $P = D_1 \oplus D_2$ . If  $D_1$  is lost, it can be recovered by decoding  $D_1 = D_2 \oplus P$ , achieving the storage efficiency of  $2/3$ , higher than  $1/2$  for 2-way replication.

**Basics of coded computation.** Recently, erasure coding has been applied to distributed computing systems to handle stragglers, called *coded computation* (CC). Specifically, a  $(k, m)$  CC splits an original computation *job* into  $k$  original *tasks* and encodes them into  $m$  parity tasks, where the above  $k + m$  tasks are then dispatched to  $k + m$  different servers to produce  $k + m$  task results. If any server becomes a straggler (i.e., failing to return its task result promptly), we can decode the task result using any  $k$  task results from  $k$  non-straggler servers. Figure 3 illustrates a CC-based matrix multiplication job  $\mathbf{Ax}$ , where  $\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 \\ \mathbf{a}_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$  and  $\mathbf{x} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ , as follows:

- Server 1 computes the original task  $\mathbf{a}_1 \mathbf{x} = [1 \ 1] \mathbf{x} = 7$ .
- Server 2 computes the original task  $\mathbf{a}_2 \mathbf{x} = [2 \ 2] \mathbf{x} = 14$ .
- Server 3 computes the parity task  $(\mathbf{a}_1 + \mathbf{a}_2) \mathbf{x} = [3 \ 3] \mathbf{x} = 21$ .

If Server 1 straggles, the system recovers the lost result through  $21 - 14 = 7$  using results from Servers 2 and 3.

**Benefits of CC.** CC offers superior cost-efficiency over replication for straggler mitigation. We evaluate this using two key metrics from existing studies [34], [55]:

- Resource efficiency [34]: Defined as  $\frac{k}{k+m}$ . For  $(k, m)$  CC,  $k$  and  $m$  are its parameters. For two-way replication,  $k = 1$  and  $m = 1$ .
- Performance-per-cost (PPC) [55]: Defined as  $\frac{T}{C}$ , where  $T$  refers to the inference throughput that is the number of original jobs divided by their processed time (measured from the traces in §V), and  $C$  refers to the monetary cost per hour that is the number of GPU instances used multiplied by the per-instance hourly price  $c$  (obtained via cloud APIs [9], [2], [22], [14]). For  $(k, m)$  CC,  $T = \frac{k}{t_1}$  and  $C = (k+m)c$ , where  $t_1$  denotes the measured time to process the  $k$  original results. For two-way replication,  $T = \frac{1}{t_2}$  and  $C = 2c$ , where  $t_2$  denotes the measured time to process the first result.

Figure 4 compares two-way replication, (2,1) CC, and (4,1) CC (all providing single-failure tolerance) under both normal (i.e., no straggler) and straggler scenarios. Each data point represents a trade-off between resource efficiency and PCC. The results reveal that: *CC dominates replication* in the trade-off space across all scenarios. Thus, many studies—including this work—focus on deploying CC in AI serving [34], [35], [59], [54] for its favorable cost-efficiency.

**State-of-the-art CC for PSSes.** Traditional CC [36], [63], [60] primarily adopt a linear-operation based simple decoder (e.g., Figure 3), which results in low accuracy for PSS workloads involving nonlinear computations (e.g., activation functions, pooling layers). Thus, recent advances [35], [59] develop *neural-based CC* that trains complex decoders based on neural network to achieve high accuracy for PSSes. As illustrated in Figure 5, the neural-based CC for PSSes:

- Encodes original jobs  $OJ_1$  and  $OJ_2$  into a parity job  $PJ$ .
- Outputs the corresponding results  $OR_1$ ,  $OR_2$ , and  $PR$ .
- Approximates the straggled result  $\hat{OR}_1$  via neural network-based decoding of available results  $OR_2$  and  $PR$ .

In this paper, we focus on applying the above neural-based CC to spot-instance PSSes for its high accuracy, but we will also observe that its complex decoding process exacerbates tail latency under preemptions, specified in §III-A.

### III. CHALLENGE AND MOTIVATION

We observe the challenge of neural-based CC applied to spot-instance PSSes via measurement analysis (§III-A), and motivate our main idea based on two findings (§III-B).

#### A. Challenge

**Measurement setting.** We conduct our measurement on a cluster composed of 32 nodes (each equipped with a 4-core CPU, a 16GiB RAM, 512GiB SSD, and an NVIDIA T4 GPU) under both normal (i.e., no straggler) and straggler scenarios. Following prior work [59], [35], we deploy two small models: i-ResNet [15], [59] and ResNet-50 [28], and two large models:

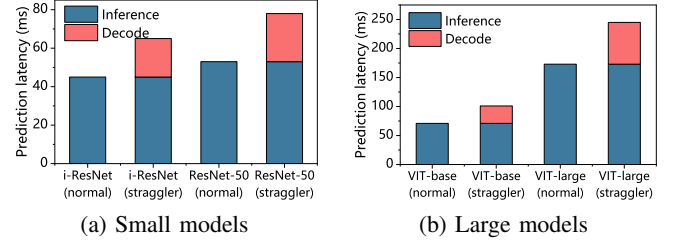


Fig. 6: Obs.#1: High decoding overhead of neural-based CC.

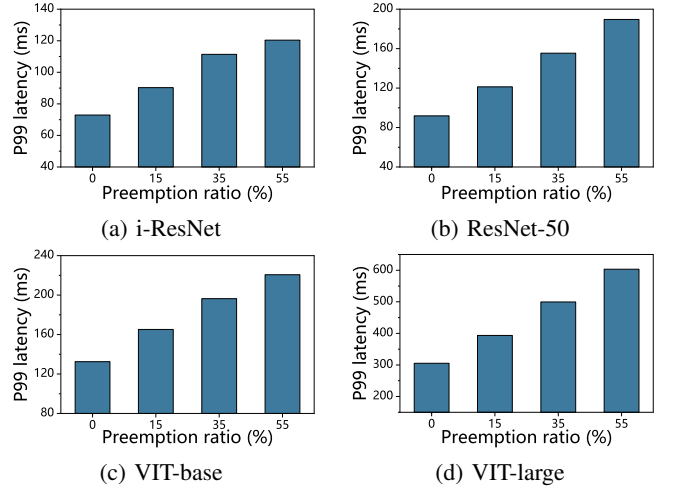


Fig. 7: Obs.#2: Prolonged tail latency under preemptions.

VIT-base [57] and VIT-large [57]; we employ two state-of-the-art neural-based CC schemes: AsymCDC [59] and Learned-CC [35], where the former is for the i-ResNet model and the latter is for the rest. We set CC parameter  $(k, m)$  as  $(2, 1)$  [59].

**Observation #1 (Neural-based CC incurs high additional decoding overhead for PSSes).** We measure the average prediction latency, which refers to the time from inputting  $k + m$  jobs to  $k + m$  instances to outputting the  $k$  original results. Note that when one instance straggles, the average prediction latency will incur the additional decoding time. Figure 6 shows that the average prediction latency with a straggler is up to 47.2% higher than that without a straggler, highlighting the high decoding overhead, which is caused by the complex decoder of neural-based CC (§II-B).

**Observation #2 (Preemptions significantly prolong the tail latency of CC-based PSSes).** We measure the P99 tail latency of 1,000 prediction jobs on instances with different *preemption ratio*, which is defined as the ratio of preempted instances to all instances within one hour [55]. We vary the preemption ratio from 0% (i.e., all instances are on-demand ones) to 55% (the worst case in prior work [20]). Figure 7 shows that the P99 latency significantly increases with the preemption ratio, since a higher ratio leads to more decoding operations, each of which also incurs high overhead as noted in Observation #1, thereby substantially prolonging the tail latency. Especially, when the preemption ratio is 55%, the P99 latency of CC-based PSSes using spot instances is up to 106.3% higher than that of those using on-demand ones.



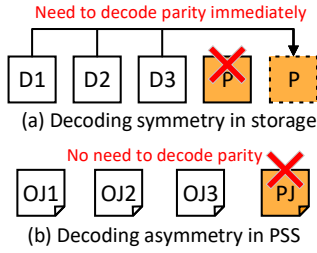


Fig. 8: Decoding difference of coded storage and PSS.

**Challenge.** Observation #1 shows the decoding inefficiency of neural-based CC, yet existing studies [32], [54], [35], [59] primarily focus on its high accuracy. This is because these works are centered on traditional straggler scenarios with extremely low straggler ratios (e.g., 0.66% [47]), where decoding operations are infrequent and thus non-critical. However, Observation #2 reveals that in spot-instance scenarios, *frequent preemptions* (e.g., 33%-55% of a cluster’s instances are preempted within an hour [20], [55]) trigger frequent decoding operations. As highlighted in Observation #1, the inefficiency of these operations significantly prolongs tail latency.

Therefore, deploying CC-based PSSes on spot instances poses a critical challenge, and our main goal in this paper is to reduce decoding operations for low latency.

#### B. Motivation

To address the challenge, we exploit two following findings.

**Finding #1 (Decoding asymmetry).** Unlike storage systems requiring uniform data/parity maintenance for reliability, CC exhibits intrinsic *decoding asymmetry* in PSSes: only original job failures necessitate decoding to recover required outputs, while parity job failures can be ignored without decoding. This is a fundamentally new finding because unlike coded storage where both original and parity units contribute equally to data reliability, PSSes only require original job outputs for correct inference, while parity jobs serve as disposable computational redundancy. Figure 8 contrasts these paradigms:

- Storage: Failed parity unit  $P$  requires immediate repair.
- PSS: Straggling parity job  $PJ$  can be safely ignored.

**Finding #2 (Preemption unevenness).** Recent studies [40], [62], [27] reveal *preemption unevenness*: the preemption ratio of spot instances varies across cloud regions, i.e., some regions are significantly prone to preemptions than others. This occurs due to temporal GPU supply imbalances across cloud regions. We analyze three open-sourced two-week AWS spot traces from three distinct regions [62], and Table II demonstrates that within one hour, the preemption ratio in Region us-east-2 is significantly higher than that in the other two regions.

**Main idea.** Building on these findings, we propose SpotCC, whose main idea is to *align preemption unevenness with decoding asymmetry* to reduce preemption-induced decoding operations. Specifically, we intentionally dispatch parity jobs to regions with high preemption ratios (*volatile regions*) and original jobs to regions with low preemption ratios (*stable*

TABLE II: Preemption ratios of different regions within one hour via AWS spot traces [62].

| Region    | Preemption ratio |
|-----------|------------------|
| us-east-2 | 56.3%            |
| us-west-1 | 0                |
| us-west-2 | 37.5%            |

*regions*), such that volatile regions avoid decoding operations for most preemptions (given parity jobs are disposable), while stable regions only need to decode few original jobs that are preempted. Figure 9 illustrates our main idea:

- Existing methods (Fig. 9(a)): (i) randomly dispatch jobs  $OJ_1, OJ_2, PJ$  to instances  $N_1, N_2, N_3$  without region awareness, (ii)  $N_2$  preemption in volatile region  $R_2$  causes  $OJ_2$  loss, and (iii) required decoding from results  $OR_1$  and  $PR$  incurs overhead.
- SpotCC (Fig. 9(b)): (i) strategically place parity job  $PJ$  on volatile instance  $N_2$ , (ii)  $N_2$  preemption only affects parity job  $PJ$ , and (iii) original results  $OR_1$  and  $OR_2$  remain available and no decoding required.

While SpotCC can benefit in reducing decoding operations caused by preemptions, the detailed design of SpotCC remains unexplored and will be specified in §IV.

#### IV. DESIGN

We design SpotCC based on the main idea in §III-B with the following design goals:

- **Effectively identifying volatile regions:** We explore intra-region preemption locality to quickly identify volatile regions for largely reducing CC decoding operations (§IV-A).
- **Enhancing volatility identification:** We explore inter-zone volatility correlation for fine-grained identification that mitigates original job congestion (§IV-B).
- **Minimizing decoding operations:** We leverage Q-learning algorithm to adaptively tune configurations for minimizing CC decoding operations (§IV-C).

##### A. Locality-based Volatility Identification: SpotCC-L

**Preemption dynamics.** To realize the core idea of SpotCC (§III-B), accurate detection of volatile and stable regions is critical. Correct classification enables strategic parity job placement in volatile regions while protecting original jobs in stable regions, effectively minimizing decoding overhead. Conversely, misclassification risks placing original jobs in volatile regions, triggering costly decoding operations.

This challenge is exacerbated by the inherent dynamics of spot instance preemptions. Figure 10(a) illustrates this using AWS spot traces [62], [4] from a 4-node p3.2xlarge GPU cluster; especially in Figure 10(b) that highlights two 3-hour time windows (also used in [55]) from the trace, preemption counts vary significantly (1 vs. 10 events). These dynamics necessitate responsive detection mechanisms that adapt to real-time preemption pattern changes.

**Insight into preemption locality.** To responsively distinguish volatile regions from stable ones, we find that preemptions exhibit strong locality in real-world spot traces [62], [40]—a pattern that can be used to indicate volatile regions.

Specifically, recent studies [40] have already revealed that in AWS spot instances, a preemption in a region is often followed by at least one more within 5 minutes—a preemption locality implying short-term volatility clustering. To characterize this volatility clustering in more detail, we further analyze

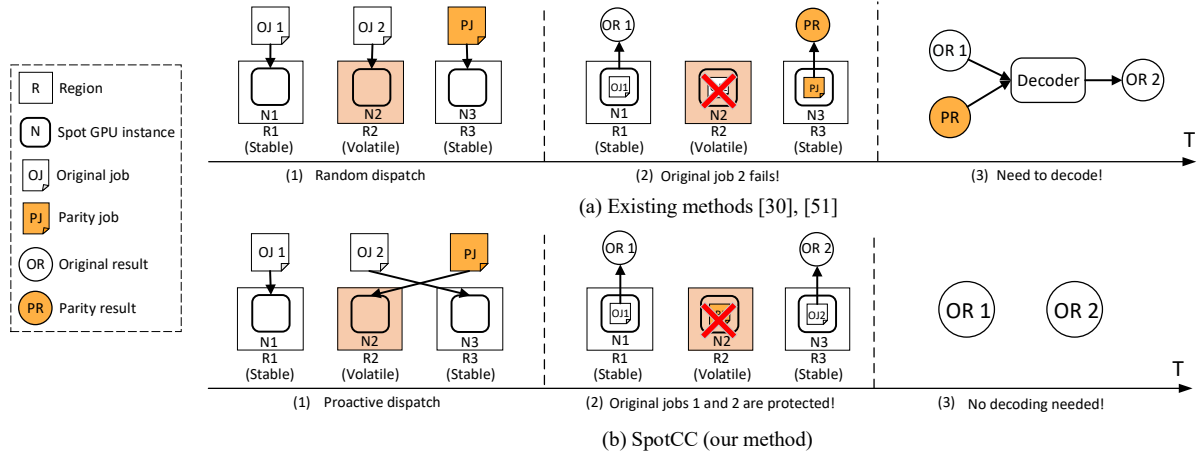


Fig. 9: Motivating Example.

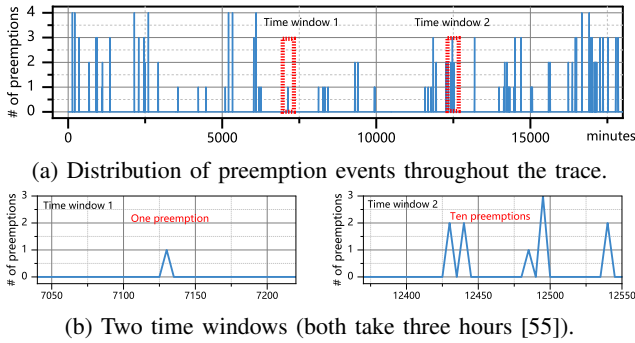


Fig. 10: Preemption dynamics in real AWS spot trace (x-axis: time in minutes; y-axis: number of preemptions per minute).

TABLE III: Preemption distribution under three different 3-hour time windows across two regions.

| Time window            | Total # of preemptions | us-east-1 | us-east-2 |
|------------------------|------------------------|-----------|-----------|
| 2023-03-12 00:00-03:00 | 13                     | 11        | 2         |
| 2023-03-14 18:30-21:30 | 10                     | 10        | 0         |
| 2023-03-15 15:00-18:00 | 12                     | 4         | 8         |

two Amazon spot traces [62] from regions *us-east-1* and *us-east-2* under different 3-hour time windows (as in [55]). As Table III shows, *us-east-1* has most preemptions in the first two time windows, while *us-east-2* dominates in the last, which shows that preemptions cluster in specific regions during specific periods, confirming volatility clustering.

**Locality-based volatility identification.** Building on this insight, SpotCC introduces SpotCC-L, a locality-aware mechanism that identifies volatile regions via preemption monitoring. The system employs dedicated *monitors* that track preemption events in real time. Upon detecting a preemption, the monitors immediately identify the affected region and mark all its instances as volatile.

Specifically, SpotCC-L operates in three sequential steps:

**Step 1: Preemption monitoring.** We deploy monitor daemons across instances for each region. Specifically, each monitor in a region maintains socket connections with all instances within the region and detects local preemption behavior via I/O

exceptions (i.e., socket timeout [55]); one frontend server then obtains the entire global preemption behavior by aggregating local behavior from all regions. To ensure fault tolerance, we adopt a 3-replica policy [23], [30]—running three daemons across different regions. If an instance hosting a monitor daemon is preempted, the system automatically relocates its daemon to maintain three active monitors.

**Step 2: Instance marking.** Upon detecting a preemption, we identify the instance’s region, mark all its remaining instances as volatile and wait for upcoming preemptions according to preemption locality. This is enabled by two `std::unordered_map` structures: `instance-to-region` and `region-to-instances` mappings, which support  $O(1)$  lookup; specifically, the former maps each instance to its corresponding region ID, and the latter maps each region to the set of instance IPs it contains. Note that a typical AI service, which often contains dozens of instances [34], [41], [59], [40], incurs negligible mapping overhead (e.g., 100 instances only incur 2.6 KB metadata overhead). We will analyze the mapping metadata overhead in §V-E.

**Step 3: Timeout mechanism.** To prevent a region from being marked as volatile permanently, we implement a timeout mechanism to revert volatile regions to stable status. Each marked region triggers a `timer`, which resets its status once a configurable threshold is reached (optimized in §IV-C).

Figure 11 illustrates SpotCC-L. When the monitor detects the preemption of instance  $N_6$  in region  $R_3$ , SpotCC-L immediately marks remaining instances like  $N_5$  in  $R_3$  as volatile. The system then dispatches original jobs ( $OJ_1$ ,  $OJ_2$ ) to stable instances ( $N_1$ ,  $N_3$ ) while directing the parity job ( $PJ$ ) to the volatile instance  $N_5$ . This design ensures that when  $N_5$  fails after  $N_6$  due to preemption locality, the original jobs remain protected without requiring decoding operations.

#### B. Fine-Grained Volatility Identification: SpotCC-F

**Limitation of SpotCC-L.** SpotCC-L employs *coarse-grained* volatility identification, which marks entire regions as volatile based on single preemptions, frequently leading to simultaneous volatility across multiple regions. Our analysis (Table IV)

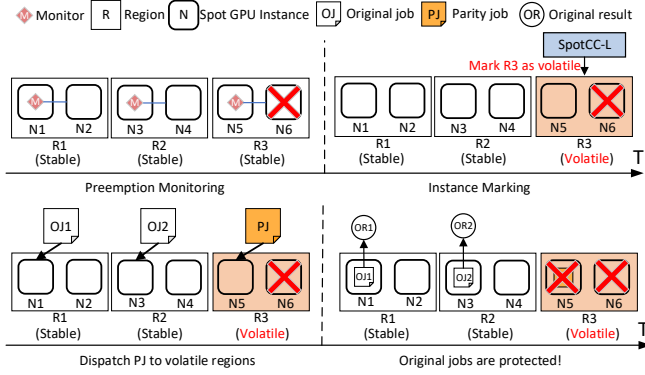


Fig. 11: Illustration of SpotCC-L.

TABLE IV: Ratio of preemptions that simultaneously occur in multiple regions, based on four different AWS traces [62].

| Trace       | AWS-1 | AWS-2 | AWS-3 | AWS-4 |
|-------------|-------|-------|-------|-------|
| Probability | 37.5% | 60.0% | 57.0% | 46.4% |

TABLE V: Inter-zone preemption correlation (via Pearson correlation coefficient) within a region and across regions.

|             | Intra-region | Across-region |
|-------------|--------------|---------------|
| Correlation | 35%-59%      | 3%-20%        |

reveals this by four real-world AWS traces, each containing three regions. Specifically, for each trace, we: (1) aggregate preemption counts of all regions, (2) apply a 3-hour sliding window, (3) count concurrent preemptions (i.e., preemptions in more than one regions per window), and (4) compute co-occurrence ratios (concurrent/total preemptions). Results in Table IV demonstrate up to a 60% probability of cross-region preemption co-occurrence, causing many regions to be marked volatile, while leaving few stable regions.

This creates critical congestion: in configurations like (2,1) CC (original jobs : parity jobs = 2:1), all original jobs congest when forced into the few remaining stable regions, causing significant queuing delays that degrade system performance.

**Insight into preemption correlation.** Modern cloud providers (AWS [4], GCP [24], Azure [42], Alibaba [1]) organize regions into *zones*, where preemptions exhibit distinct correlation (Table V), quantified via Pearson correlation coefficient [58], [40]:

- *Intra-region correlation*: High preemption correlation (35-59%) of different zones within the same region.
- *Cross-region correlation*: Minimal preemption correlation (3-20%) of different zones across different regions.

This insight motivates our shift to intra-region volatility marking of different zones: a fine-grained zone-level approach, which addresses the limitation of SpotCC-L's coarse-grained region-level method (i.e., marking a region's zones altogether).

**Fine-grained volatility identification.** Building on this insight, SpotCC introduces SpotCC-F, a fine-grained volatility identification mechanism that leverages preemption correlation between preemptions in different zones within the same region to identify volatility at the zone level. The mechanism marks two types of zones: (i) the zone where the preemption occurs,

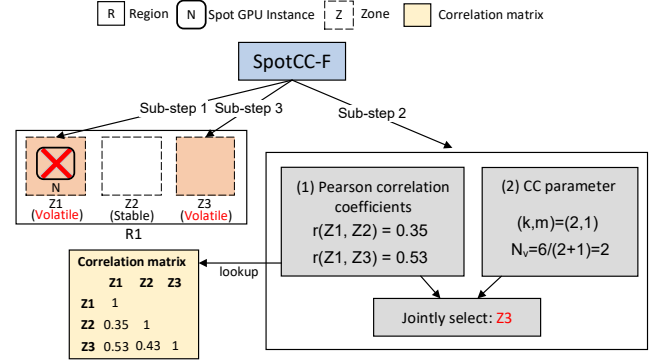


Fig. 12: Illustration of SpotCC-F.

and (ii) other zones within the same region that have the strongest correlation with it. This leaves more zones (marked as stable) within the region available to host original jobs, effectively mitigating their congestion.

SpotCC-F retains SpotCC-L's preemption monitoring (Step 1) and timeout mechanism (Step 3), with modifications exclusively on Step 2, now featuring three key sub-steps:

**Sub-step 1: Marking the preempted zone.** Upon detecting a preempted instance, the system immediately identifies and marks its residing zone as volatile.

**Sub-step 2: Selecting the most correlated zones.** While prior work [58], [40] uses Pearson correlation coefficient solely for zone analysis, it provides no principled approach to selecting the optimal number of zones to mark. To address this limitation, we propose a Pearson-CC-joint selection scheme that:

- Quantifies inter-zone relationships via Pearson coefficients.
- Determines the marking threshold by CC parameters (k,m).

Specifically, we first quantify zone correlation through pairwise Pearson correlation analysis of historical preemption logs, sampled at fixed intervals (e.g., 5 minutes [62]). Given two zone logs  $X = [x_1, \dots, x_n]$  and  $Y = [y_1, \dots, y_n]$ , where  $x_i, y_i$  represent preemption counts at the  $i^{th}$  timestamp, the correlation of two zones is:

$$r(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (2)$$

where  $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$  and  $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ .

Further, to accelerate the computation of Pearson correlation, we maintain an in-memory *correlation matrix* using `std::vector` to store inter-zone correlation values, enabling  $O(1)$  correlation lookups between zones. For typical AI deployments with  $\sim 10$  zones [62], [40], this introduces negligible overhead (e.g.,  $\sim 1.5$ KB memory for 10 zones). A detailed overhead analysis appears in §V-E.

Next, given a  $(k,m)$  CC scheme with an original-to-parity job ratio of  $k:m$ , to avoid the congestion of original jobs, we enforce the ratio of stable zones to volatile zones to be at least  $k:m$ . Thus, we determine the maximum number of zones that can be marked volatile (denoted by  $N_v$ ) as:

$$N_v = \left\lfloor \frac{m}{k+m} N_z \right\rfloor \text{ where } N_z \text{ is total number of zones.} \quad (3)$$

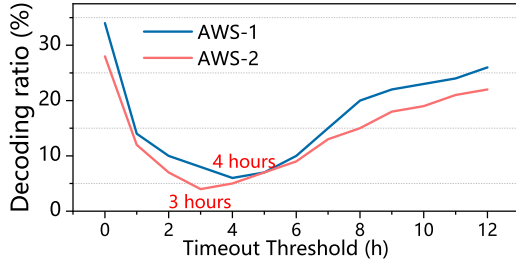


Fig. 13: Decoding ratio with timeout threshold.

Since we have marked one zone that has the preemption as volatile (Sub-step 1), we only need to select  $N_v - 1$  additional zones that can be marked volatile. The selection process first ranks all zones by their correlation values with the preempted zone computed via Equation (2), then selects the top  $N_v - 1$  candidates.

**Sub-step 3: Marking the selected zones.** Finally, we mark the most correlated zones selected in Sub-step 2 as volatile.

Figure 12 illustrates SpotCC-F, which first marks the zone  $Z_1$  (where the preemption occurs) volatile. Then SpotCC-F queries the correlation matrix to obtain the correlation values between the  $Z_1$  and other zones ( $Z_2$  and  $Z_3$ ). Assume  $(k, m) = (2, 1)$ ; the maximum number of volatile zones can be computed as  $N_v = \lfloor \frac{m}{k+m} N_z \rfloor = 2$ . Since  $Z_1$  has been marked in Sub-step 2-1, we can mark one additional zone. Since  $Z_3$  is the most correlated zone with  $Z_1$  ( $r(Z_1, Z_3) = 0.53 > r(Z_1, Z_2) = 0.35$ ), we mark  $Z_3$  as volatile.

#### C. Adaptive Profiling-Guided Timeout: SpotCC-A

**Limitation of SpotCC-F.** The effectiveness of SpotCC-F critically depends on properly setting the volatile zone timeout threshold. Our analysis reveals a U-shaped relationship between timeout threshold and decoding overhead. Specifically, excessively short thresholds prematurely mark volatile zones as stable, causing original jobs assigned to them to be preempted and trigger decoding. Conversely, overly long thresholds leave many volatile zones marked as such even after they turn stable, preventing new preempted zones from being marked as volatile (due to the maximum limit in Equation (3)). In this case, original jobs assigned to these newly preempted zones are also preempted, triggering decoding.

To demonstrate the U-shaped relationship, we conduct a measurement analysis (with the same settings as in §V) to measure the *decoding ratio*, defined as the proportion of original jobs requiring decoding to all original jobs. This ratio reflects the decoding overhead, with a lower decoding value indicating fewer decoding operations. Figure 13 shows the AWS-1 trace achieves the lowest decoding ratio at a timeout of 4 hours, while AWS-2 does so at 3 hours. This variation highlights the need for adaptive timeout threshold tuning.

**Adaptive profiling-guided timeout tuning.** To adaptively tune the timeout threshold, reinforcement learning (RL) is widely used for this purpose [33]. However, determining an appropriate initial value remains unexplored—poor initialization can expand the search space and slow convergence.

#### Algorithm 1 Q-learning Algorithm

---

```

1: Set initial state  $s$  based on profiling.
2: Initialize Q-table  $Q(s, a)$  arbitrarily
3: Set learning rate  $\alpha$ , discount factor  $\gamma$ , exploration rate  $\epsilon$ 
4: while not terminated do
5:   if rand() <  $\epsilon$  then Choose action  $a$  randomly
6:   else Choose action  $a$  with the largest  $Q(s, a)$ 
7:   end if
8:   Apply timeout adjustment based on action  $a$ 
9:   Run system for one interval and observe reward  $r$ 
10:  Observe next state  $s'$ 
11:  Update Q-value:
      
$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

12:   $s \leftarrow s'$ 
13: end while

```

---

To address this, SpotCC introduces SpotCC-A, an adaptive profiling-guided RL scheme that first profiles an appropriate default timeout threshold from historical preemption logs, and then dynamically adjusts it via RL, specified as follows:

**Step 1: History-aware profiling.** We analyze historical preemption logs (recorded at 5-minute intervals) to compute the average duration of volatile periods—continuous windows with preemptions—which we use as our initial timeout threshold. Volatile period identification works as follows:

- *Period Initiation:* A new volatile period begins at the first timestamp with preemptions.
- *Period Extension:* The period remains active if subsequent preemptions occur within rolling 3-hour windows ([55]); each new preemption resets the 3-hour window timer.
- *Period Termination:* The period concludes when the current 3-hour window elapses without preemptions.

The algorithm iterates through the log, records the duration of each volatile period, and outputs the average duration as the initial timeout threshold.

**Step 2: RL-based adaptive tuning.** We formulate timeout adjustment as a RL problem with the following components: (i) *State:*  $s = (T, DR)$ , capturing the current timeout threshold ( $T$ ) and decoding ratio ( $DR$ ); (ii) *Action:*  $a \in \{-0.5h, 0, +0.5h\}$ , discrete threshold adjustments; (iii) *Reward:*  $r = -DR$ , incentivizing decoding ratio minimization. The RL agent iteratively observes states, applies adjustments, and receives feedback to learn an optimal threshold policy.

We implement this model using Q-learning [61] for its computational efficiency, maintaining a Q-table that maps state-action pairs  $(s, a)$  to expected rewards. To balance exploration and exploitation, we employ an  $\epsilon$ -greedy policy, which chooses the action with the highest reward or a random action with an exploration rate  $\epsilon$  ( $\epsilon = 0.1$  [33], [43]).

Algorithm 1 presents the Q-learning implementation with three initialization steps: (i) state  $s$  (Line 1), (ii) Q-table with random values (Line 2), and (iii) hyperparameters  $\alpha$ ,  $\gamma$ ,  $\epsilon$  (Line 3). During execution, it selects action  $a$  randomly (with



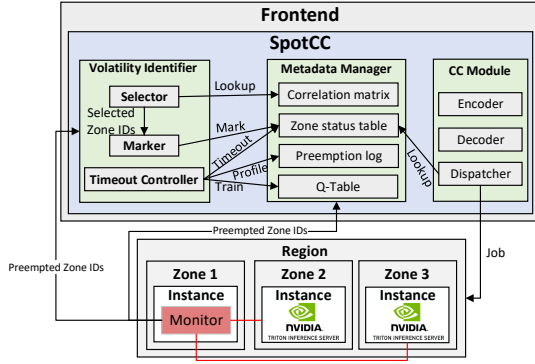


Fig. 14: Architecture of SpotCC.

probability  $\epsilon$ , Line 5) or greedily (by maximizing  $Q(s, a)$ , Line 6). After applying the timeout adjustment via  $a$  (Line 8), it calculates reward  $r = -DR$  (Line 9), observes the new state  $s'$  (Line 10), updates  $Q(s, a)$  via Q-learning rule (Line 11), and transitions to  $s'$  for continued training (Line 12).

After Algorithm 1 converges (i.e., the Q-values stabilize for all states), we can obtain the optimized timeout threshold  $T$ , achieving the minimal decoding ratio  $DR$ .

## V. EVALUATION

### A. Implementation

We prototype SpotCC in C++ (~8.5K LoCs) atop NVIDIA Triton Inference Server [44], a popular inference engine [40], [41]. Figure 14 shows the prototype architecture comprising four core modules: Metadata Manager, Volatility Identifier and CC Module (at frontend), and Monitor (at instances).

**Metadata Manager.** It maintains four key data structures for volatility identification: (i) a *correlation matrix* storing pairwise zone correlation values; (ii) a *zone status table* tracking volatile/stable states; (iii) *historical preemption logs* recording zone-level preemption counts over 24 hours to capture preemption trend; (iv) a *Q-Table* for Q-learning values. It continuously updates preemption logs using zone IDs from Monitor, and periodically updates the correlation matrix using recent preemption logs.

**Volatility Identifier.** It detects volatile zones via three coordinated components: (i) a *selector* that queries the correlation matrix using preempted zone IDs and applies the Pearson-CC-joint scheme to determine zones for marking; (ii) a *marker* that updates zone status in the status table; (iii) a *timeout controller* that manages marking duration-initially set through historical log profiling and dynamically adjusted via the Q-Table.

**CC Module.** It is based on LibTorch [48] that supports various types of CC, including both simple linear CC schemes [34], [59] and complex neural-based CC schemes [35], [59].

**Monitor.** It sends preempted zone IDs to Volatility Identifier and Metadata Manager via efficient gRPC protocol [25].

### B. Experiment Setup

**Traces.** Following established methodology [55], [20], [41], [40], we use replayed spot instance traces on on-demand

TABLE VI: Models used for evaluation.

| Model     | Size    | Params |
|-----------|---------|--------|
| i-ResNet  | 24 MB   | 6.3M   |
| ResNet-50 | 90 MB   | 25.6M  |
| VGG-16    | 512 MB  | 138.4M |
| ViT-L     | 1.13 GB | 307.1M |
| ViT-H     | 2.35 GB | 630.8M |

infrastructure for reproducibility. We analyze two real-world AWS traces [62], [40]:

- **AWS-1:** 2-week trace across 3 regions (12 instances total), replayed on Alibaba Cloud’s `ecs.gn6i-c4g1.2xlarge` instances (8-core CPU, 31GiB RAM, 4TiB SSD, T4 16GiB).
- **AWS-2:** 2-month trace with 3 regions  $\times$  3 zones  $\times$  4 instances (36 total), using identical hardware configuration.

**Workloads.** We use two workloads: (i) *MAF*: the Microsoft Azure Functions trace [50] (for real-world scenarios); (ii) *Poisson*: a synthetic workload [34], [40] with Poisson-distributed request arrivals (for sensitivity analysis). This combination covers both realistic and controlled conditions.

**Models.** We use five representative DNN models spanning different scales: (i) Small models: i-ResNet [15], [59], ResNet-50 [28], VGG-16 [53], and (ii) Large models: ViT-L and ViT-H [57]. Model specifications are detailed in Table VI. Following established benchmarks [34], [41], we test batch sizes  $\{1, 2, 4, 8\}$  and adopt  $(k, m)=(2, 1)$  as the default CC parameter as it represents the most common configuration [34], [35], [59]. We also conduct sensitivity analysis of batch size and varying  $k$  (Experiments 3 and 4).

**Comparison.** We compare against: (1) *NeuralCC*, representing state-of-the-art neural-based CC methods (Learned-CC [35] and AsymCDC [59]), and (2) an *Omniscient* oracle that uses complete future preemption traces [16], [40] to perfectly avoid decoding (establishing an ideal upper bound).

### C. Main Experiment Results

**Experiment 1 (Latency).** We send ~10K prediction jobs to the system and measure the average, P90, P95, and P99 latency among NeuralCC, SpotCC, and Omniscient schemes on AWS-1 and AWS-2 across five deployed models. Figure 15 shows the results:

(i) Compared to NeuralCC, SpotCC significantly reduces the tail latency due to the large reduction of decoding operations. Taking P99 latency as an example, SpotCC outperforms NeuralCC by up to 83.9% on AWS-1 for the i-ResNet model. This improvement stems from SpotCC’s core innovation: greatly reducing decoding operations by matching decoding asymmetry and preemption unevenness. Thus, SpotCC facilitates deploying neural-based CC to PSSes on spot instances.

(ii) Compared to Omniscient, SpotCC achieves near-optimal performance, close to Omniscient, indicating that most decoding operations are eliminated despite lacking full future knowledge. This confirms the effectiveness of our timeout optimization and volatility identification mechanisms.

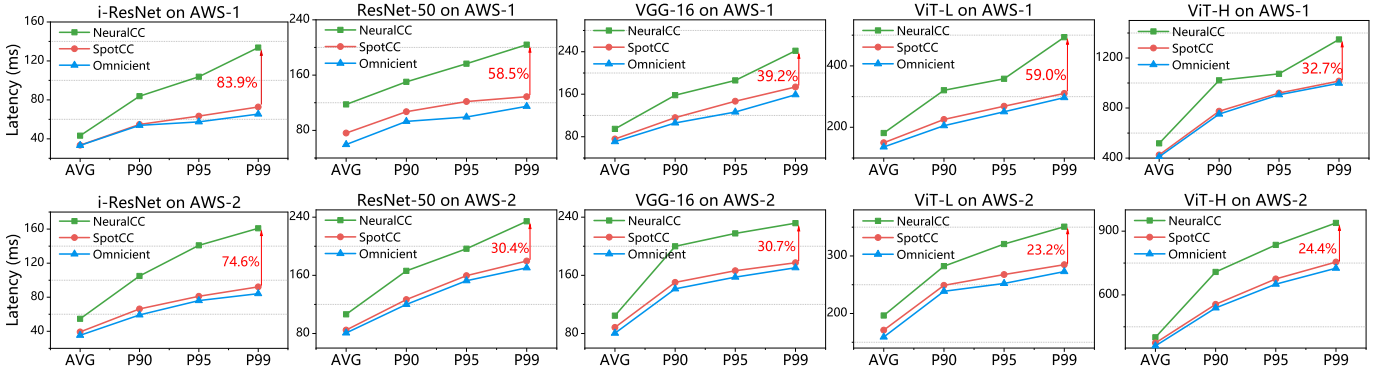


Fig. 15: Average, P90, P95, P99 latency among NeuralCC, SpotCC, and Omnicient (Exp. 1).

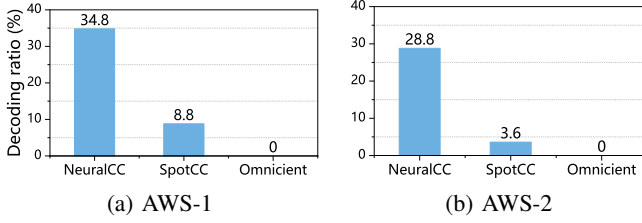


Fig. 16: Comparison of decoding ratio (Analysis of Exp. 1).

(iii) The latency improvements are more pronounced for smaller models (i-ResNet, ResNet-50, and VGG-16) than for large models (ViT-L and ViT-H), as the smaller models' decoding overhead constitutes a larger proportion of their shorter inference latency. Nevertheless, SpotCC still outperforms NeuralCC by up to 59.0% even for large models, demonstrating consistent benefits across model scales.

**Analysis of Experiment 1.** We further measure the decoding ratio (introduced in §IV-C) to verify that the performance gain of SpotCC over NeuralCC stems from the reduced decoding overhead. Figure 16 shows that compared to NeuralCC, SpotCC significantly reduces the average decoding ratio by 74.7% in AWS-1 and 87.5% in AWS-2. Compared to Omnicient, on both traces, SpotCC is close to Omnicient's optimal decoding ratio (i.e., 0). This reveals why SpotCC can greatly reduce tail latency compared to NeuralCC and has a similar performance as the Omnicient.

**Experiment 2 (Ablation study).** We measure the P99 latency of NeuralCC, SpotCC-L, SpotCC-F and SpotCC-A on AWS-1 and AWS-2 across one small model (i-ResNet) and one large model (ViT-H), to separately evaluate the performance gain by each scheme. Figure 17 shows the results:

- (i) Compared to NeuralCC, SpotCC-L achieves significantly lower P99 tail latency (as low as  $0.62\times$  on AWS-1 under the small model), as it largely reduces CC decoding operations.
- (ii) Compared to SpotCC-L, SpotCC-F shows a slight performance improvement on AWS-1. This is because each region in AWS-1 contains only one zone, limiting SpotCC-F's ability to leverage its fine-grained zone-level design. By contrast, on AWS-2, SpotCC-F demonstrates a more significant advantage over SpotCC-L. Since each region in AWS-2 consists of three zones, SpotCC-F can fully utilize its zone-level granularity to

more accurately identify volatile zones.

(iii) Across both traces, SpotCC-A consistently reduces P99 latency compared to SpotCC-F (12.5% on average). This improvement stems from SpotCC-A's ability to minimize decoding overhead via adaptively tuning the timeout threshold.

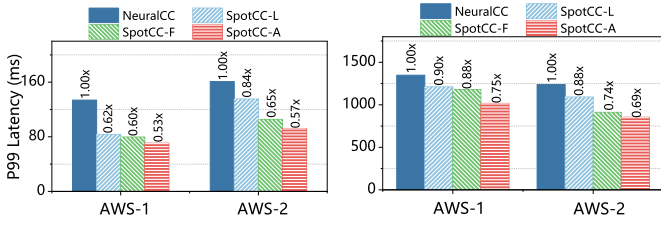
#### D. Sensitivity and Scalability

**Experiment 3 (Sensitivity to  $k$ ).** We vary the CC parameter  $k$  from  $\{2, 4, 6, 8\}$  to analyze its impact on SpotCC's performance. Figure 18 shows the results:

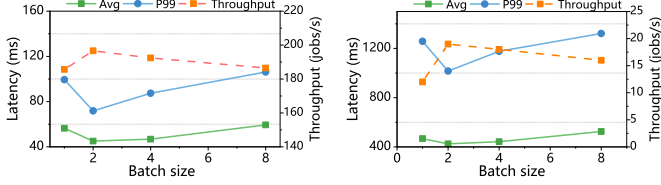
- (i) Compared to NeuralCC, as  $k$  increases, SpotCC always maintains superior performance, and NeuralCC's P99 latency rises significantly faster than that of SpotCC. This is because a CC decoder with a larger  $k$  requires more original jobs to decode, thus raising decoding overhead. In addition, NeuralCC's P99 latency increases with  $k$  due to its large number of decoding operations; in contrast, SpotCC exhibits only a slight increase, as it greatly reduces the number of decoding operations, thus effectively mitigating the impact of larger  $k$ .
- (ii) Compared to Omnicient, the P99 latency gap between SpotCC and Omnicient grows with  $k$ . This is because Omnicient does not require decoding, so its tail latency remains unaffected by larger  $k$ , while SpotCC still incurs some decoding overhead, which increases with  $k$ , thereby widening the gap as  $k$  grows. Nevertheless, CC's common parameter settings often adopt small  $k$  for practical efficiency (e.g.,  $k = 2$ ) [34], [59].

**Experiment 4 (Sensitivity to batch size).** We study the impact of batch size when  $k = 2$ . Figure 19 shows the average latency, P99 latency, and throughput (i.e., prediction jobs processed per second) of SpotCC for batch sizes  $\{1, 2, 4, 8\}$ . We observe that both average and P99 latency initially decrease then increase, while throughput shows the opposite trend: first rising then declining. Interestingly, both latency and throughput metrics reach their optimal point when the batch size equals  $k$  (batch size =  $k = 2$ ). This is because CC, by nature of its encoding, requires waiting for  $k$  jobs to encode together—aligning well with the batching mechanism, where inference also waits for a batch of a given size. Thus, when batch size equals  $k$ , SpotCC achieves optimal performance in average, P99 latency, and throughput.

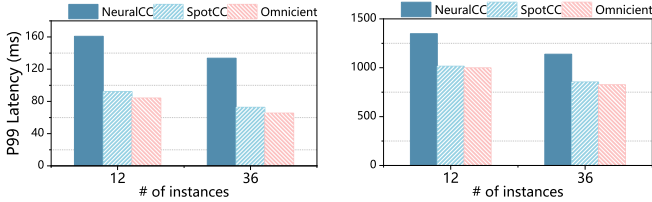
**Experiment 5 (Sensitivity to workloads).** We evaluate SpotCC under the Microsoft Azure Functions (MAF) and



(a): Small model (i-ResNet) (b): Large model (ViT-H)  
Fig. 17: Ablation study (Exp. 2).



(a): Small model (i-ResNet) (b): Large model (ViT-H)  
Fig. 19: Sensitivity to batch size (Exp. 4).



(a) Small model (i-ResNet) (b) Large model (ViT-H)

Fig. 21: Scalability (Exp. 6).

TABLE VII: Monetary cost (Exp. 7).

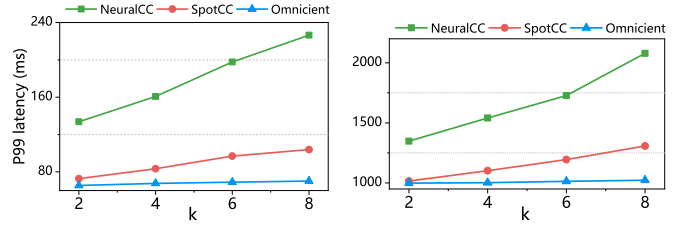
|                     | Two-way Replication | (2,1) CC     |
|---------------------|---------------------|--------------|
| # of required inst. | 40                  | 30           |
| cost on on-demand   | 55.48 (\$/h)        | 41.14 (\$/h) |
| cost on spot        | 13.8 (\$/h)         | 9.9 (\$/h)   |

Poisson workloads, where MAF is fluctuating and bursty while Poisson is relatively stable. Figure 20 shows the average and P99 latency of NeuralCC, SpotCC, and Omnicient.

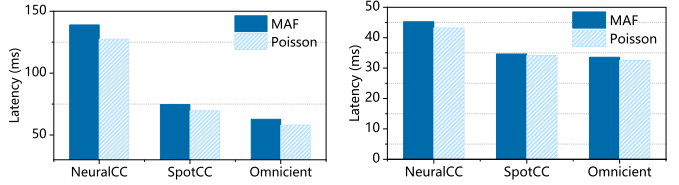
(i) Compared to the Poisson workload, all three schemes exhibit larger average and P99 latency under MAF. This occurs because MAF includes bursty periods that create sudden spikes in inference pressure, leading to increased latency.

(ii) Under both workloads, SpotCC consistently outperforms NeuralCC and is close to Omnicient in performance, which shows its robustness across diverse request arrival patterns.

**Experiment 6 (Scalability).** Our experiments demonstrate SpotCC's decent scalability across 12-36 instances. As shown in Figure 21, SpotCC maintains consistent P99 latency reduction over NeuralCC (average 38% improvement) regardless of instance count, since its performance gain stems primarily from decoding ratio reduction rather than scale. Notably, SpotCC achieves near-Omnicient performance (within 8-12%) at all scales. P99 latency decreases with additional instances due to increased computational resources and higher concurrency. These results confirm that SpotCC's advantages are preserved while benefiting from horizontal scaling.



(a): Small model (i-ResNet) (b): Large model (ViT-H)  
Fig. 18: Sensitivity to  $k$  (Exp. 3).



(a): P99 latency (b): Average latency

Fig. 20: Sensitivity to workload (Exp. 5).

TABLE VIII: Metadata overhead (Exp. 8).

| Metadata            | # of inst = 12 | # of inst = 36 |
|---------------------|----------------|----------------|
| Region instance map | 0.7 KB         | 1.7 KB         |
| Preemption log      | 3.5 KB         | 10.3 KB        |
| Correlation matrix  | 0.17 KB        | 0.5 KB         |
| Q-Table             | 550 KB         | 550 KB         |
| <b>Total</b>        | <b>~560 KB</b> | <b>~570 KB</b> |

## E. Cost and Metadata Analysis

**Experiment 7 (Monetary cost).** We analyze the monetary cost via Alibaba Cloud Pricing API [2] to confirm the cost-efficiency of CC on spot instances. Table I compares (2,1) CC against two-way replication (both providing single-failure tolerance) for serving 20 jobs under on-demand and spot instances. First, while replication requires 40 instances (20 original + 20 replicas), CC uses only 30 (20 original + 10 parity), reducing the instance count by 25%, e.g., 55.48 USD/h for replication vs. 41.14 USD/h for CC on on-demand instances. Next, CC on spot instances can further reduce monetary cost significantly over CC on demand, by over 75% (e.g., 9.9 USD/h vs. 41.14 USD/h), achieving ultra-low monetary cost.

**Experiment 8 (Metadata overhead).** We measure both the total metadata overhead of SpotCC and the overhead of key metadata components separately, including region instance maps (§IV-A), preemption logs (§IV-B, §IV-C), correlation matrices (§IV-B), and Q-tables (§IV-C). Table VIII shows that the total metadata overhead is on the KB scale, which is trivial for modern systems. Notably, the Q-table is the primary source of metadata overhead, while other components contribute minimally. Importantly, the size of the Q-table depends only on the state space size, not the number of instances. Thus, the overall metadata overhead remains small in size even as the number of instances increases.

## F. Limitation Analysis

Note that the above experiments are all conducted using existing real-world traces, which exhibit two key features:

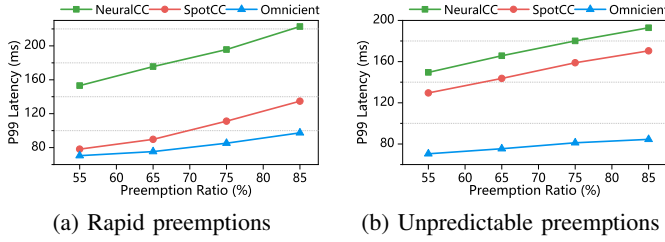


Fig. 22: Limitation analysis (Exp. 9 & 10).

(i) *low preemption ratio*: an average preemption ratio of 33%–55% [55], [20]; and (ii) *predictable preemptions*: preemptions exhibit locality, which can be leveraged for preemption prediction. SpotCC’s core design is built upon these two features. Thus, when a trace lacks these characteristics (e.g., rapid or unpredictable preemptions), SpotCC’s performance gains may diminish.

To investigate SpotCC’s limitations in scenarios lacking these two features, we specifically generate two additional traces based on the existing real-world traces:

- *Rapid-preemption trace*: We first replicate the real-world AWS-1 trace (e.g., same duration and number of zones) and measure its preemption ratio. We then inject additional preemption events to raise the ratio to 55–85%, which exceeds the typical range reported in prior studies [55], [20] and thus reflects rapid preemption behavior.
- *Unpredictable-preemption trace*: We first construct a preemption-free trace (no preemption events) matching AWS-1’s configuration (e.g., same duration and number of zones). We then count the preemption events in the original AWS-1 trace and randomly inject an equal number into the preemption-free trace, thereby eliminating temporal locality and creating unpredictable preemptions..

We evaluate SpotCC on these two synthetic traces to characterize SpotCC’s limitations.

**Experiment 9 (Rapid preemptions).** Figure 22(a) shows the P99 latency of NeuralCC, SpotCC and Omniscient schemes under rapid preemption patterns. Compared to Omniscient, we see that SpotCC’s performance degrades more significantly as the preemption rate increases (at a 55% preemption ratio, SpotCC’s latency is only 11.0% higher than Omniscient’s; at 85%, it exceeds Omniscient by 38.3%). The main reason is that SpotCC adopts a *reactive* design—responding only after preemption detection. When numerous preemptions occur in a short period, SpotCC cannot adapt quickly enough. To better handle such scenarios, SpotCC would need to incorporate more *proactive* mechanisms in the future, enabling it to predict preemptions in advance and respond promptly.

**Experiment 10 (Unpredictable preemptions).** Figure 22(b) shows the P99 latency of NeuralCC, SpotCC and Omniscient schemes under unpredictable preemption patterns. We see that SpotCC performs similarly to NeuralCC (improving P99 latency by only 14% on average compared to NeuralCC). This is because SpotCC heavily relies on preemption locality, a key feature of real-world traces. When we intentionally disrupt

this locality, SpotCC can no longer effectively avoid decoding operations. To address such scenarios, SpotCC would need to leverage additional signals (e.g., network fluctuations or CPU activity) to identify preemptions, rather than relying solely on preemption locality.

## VI. RELATED WORK

**Coded Computation for Prediction Serving.** Coded computation (CC) for prediction serving has evolved significantly from its initial replication-based approaches. ParM [34] first introduces learning-based CC to prediction serving systems, though with inherent accuracy and scalability limitations. ApproxIFER [54] advances the field through approximate computation capable of handling more stragglers. Learned-CC [35] subsequently improves accuracy using neural-based encoder-decoder pairs, albeit with increased coding latency. Most recently, AsymCDC [59] achieves an optimal balance between accuracy and latency through its asymmetric framework combining linear encoders with neural decoders. However, all existing works target on-demand instances, leaving the cost-efficiency potential of spot instances unexplored. Our work establishes the first CC framework specifically designed for spot instances while simultaneously solving the fundamental CC decoding inefficiency problem in volatile environments.

**AI Serving on Spot Instances.** The growing adoption of AI serving has spurred significant interest in leveraging spot instances for cost reduction. Prior work includes: Cocktail [26] expanding ensemble models via spots (with occasional accuracy drops), MArk [64] overcoming over-provisioning in ML serving through SLO-aware scaling, SpotServe [41] pioneering LLM serving on spots via dynamic parallelism, optimized migration and stateful recovery, and SkyServe [40] ensuring high availability through over-provisioning multi-region cheap spot replicas, and dynamically falling back to on-demand ones. Our work introduces a new dimension to save deployment cost by facilitating CC for spot-instance prediction serving.

## VII. CONCLUSION

We propose SpotCC, the first framework enabling cost-efficient coded computation for prediction serving on spot instances, which addresses the fundamental challenge of decoding inefficiency under frequent preemptions. We design locality-aware volatility detection to reduce unnecessary decoding, fine-grained zone-level identification to mitigate job congestion, and adaptive timeout tuning to dynamically minimize decoding operations. Cloud experiments show that SpotCC achieves significant improvements in tail latency while drastically reducing deployment costs, demonstrating its effectiveness for cost-sensitive and latency-critical prediction serving on spot instances.

## ACKNOWLEDGMENT

This work was supported by the National Key R&D Program of China (No. 2022YFB4501300), the National Natural Science Foundation of China (No. 62272185), and Key Laboratory of Information Storage System Ministry of Education of China. The corresponding author is Yuchong Hu.



## REFERENCES

- [1] Alibaba Cloud. <https://www.aliyun.com/>.
- [2] Alibaba Cloud Pricing. <https://cn.aliyun.com/price>.
- [3] Amazon. Google Lens: real-time answers to questions about the world around you. <https://developer.amazon.com/en-US/alexa>.
- [4] Amazon. Machine learning on aws. <https://aws.amazon.com/machine-learning/>.
- [5] Amazon Alexa. <https://developer.amazon.com/en-US/alexa>.
- [6] Apple. iOS Siri. <https://www.apple.com/ios/siri/>.
- [7] AWS. Load Balancer of AWS. <https://docs.aws.amazon.com/elasticloadbalancing/latest/classic/config-idle-timeout.html>.
- [8] AWS Cloudwatch. <https://aws.amazon.com/cn/cloudwatch/>.
- [9] AWS Pricing. <https://aws.amazon.com/pricing>.
- [10] AWS S3. <https://aws.amazon.com/cn/s3/>.
- [11] AWS spot instance advisor. <https://aws.amazon.com/cn/ec2/spot/instance-advisor/>.
- [12] AWS spot instance termination notice. <https://aws.amazon.com/about-aws/whats-new/2015/01/05/announcing-amazon-ec2-spot-instance-termination-notices/>.
- [13] AWS Spot Instances. <https://www.amazonaws.cn/ec2/spot-instances/>.
- [14] Azure Pricing. <https://azure.microsoft.com/en-us/pricing>.
- [15] J. Behrmann, W. Grathwohl, R. T. Chen, D. Duvenaud, and J.-H. Jacobsen, "Invertible residual networks," in *Proc. of ICML*, 2019.
- [16] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [17] Cost of running GPT-3. [https://www.ri.se/en/news/blog/generative-ai-does-not-run-on-thin-air?utm\\_source=chatgpt.com](https://www.ri.se/en/news/blog/generative-ai-does-not-run-on-thin-air?utm_source=chatgpt.com).
- [18] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A Low-Latency online prediction serving system," in *Proc. of USENIX NSDI*, 2017.
- [19] T. Dinh and K. Lee, "Coded-inet for resilient prediction serving systems," in *Proc. of ICML*. PMLR, 2021.
- [20] J. Duan, Z. Song, X. Miao, X. Xi, D. Lin, H. Xu, M. Zhang, and Z. Jia, "Parcae: Proactive, Liveput-Optimized DNN training on preemptible instances," in *Proc. of USENIX NSDI*, 2024.
- [21] E. Efatinasab, F. Marchiori, A. Brighente, M. Rampazzo, and M. Conti, "Faultguard: a generative approach to resilient fault prediction in smart electrical grids," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2024, pp. 503–524.
- [22] GCP Pricing. <https://cloud.google.com/pricing>.
- [23] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The google file system," in *Proc. of ACM SOSP*, 2003, pp. 29–43.
- [24] Google Cloud AI. <https://cloud.google.com/products/machine-learning/>.
- [25] gRPC. <https://grpc.io/>.
- [26] J. R. Gunasekaran, C. S. Mishra, P. Thinakaran, B. Sharma, M. T. Kandemir, and C. R. Das, "Cocktail: A multidimensional optimization for model serving in cloud," in *Proc. of USENIX NSDI*, 2022.
- [27] A. Harlap, A. Chung, A. Tumanov, G. R. Ganger, and P. B. Gibbons, "Tributary: spot-dancing for elastic services with latency SLOs," in *Proc. of USENIX ATC*, 2018.
- [28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. of IEEE CVPR*, 2016.
- [29] How To Reduce Cold Start Times For LLM Inference. <https://scale.com/blog/reduce-cold-start-time-llm-inference>.
- [30] C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li, and S. Yekhanin, "Erasure coding in windows azure storage," in *Proc. of USENIX ATC*, 2012.
- [31] T. Jahani-Nezhad and M. A. Maddah-Ali, "Berrut approximated coded computing: Straggler resistance beyond polynomial computing," *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 2022.
- [32] M. Karami, D. Schuurmans, J. Sohl-Dickstein, L. Dinh, and D. Duckworth, "Invertible convolutional flow," *NeurIPS*, vol. 32, 2019.
- [33] Y. G. Kim and C.-J. Wu, "Autoscale: Energy efficiency optimization for stochastic edge inference using reinforcement learning," in *Proc. of MICRO*. IEEE, 2020.
- [34] J. Kosaian, K. Rashmi, and S. Venkataraman, "Parity models: erasure-coded resilience for prediction serving systems," in *Proc. of ACM SOSP*, 2019.
- [35] J. Kosaian, K. Rashmi, and S. Venkataraman, "Learning-based coded computation," *IEEE Journal on Selected Areas in Information Theory*, 2020.
- [36] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," *IEEE Trans. on Information Theory*, 2017.
- [37] K. Lee and M. Son, "Deepspotcloud: leveraging cross-region gpu spot instances for deep learning," in *Proc. of IEEE CLOUD*. IEEE, 2017.
- [38] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez *et al.*, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *Proc. of USENIX OSDI*, 2023.
- [39] Loading Llama-2 70b 20x faster. <https://www.anyscale.com/blog/loading-llama-2-70b-20x-faster-with-anyscale-endpoints>.
- [40] Z. Mao, T. Xia, Z. Wu, W.-L. Chiang, T. Griggs, R. Bhardwaj, Z. Yang, S. Shenker, and I. Stoica, "Skyserve: Serving ai models across regions and clouds with spot instances," in *Proc. of EuroSys*, 2025.
- [41] X. Miao, C. Shi, J. Duan, X. Xi, D. Lin, B. Cui, and Z. Jia, "Spotserve: Serving generative large language models on preemptible instances," in *Proc. of ACM ASPLOS*, 2024.
- [42] Microsoft. Azure machine learning studio. <https://azure.microsoft.com/en-us/services/machine-learning-studio/>.
- [43] R. Nishtala, P. Carpenter, V. Petrucci, and X. Martorell, "Hipster: Hybrid task manager for latency-critical cloud workloads," in *Proc. of IEEE HPCA*. IEEE, 2017.
- [44] NVIDIA. Triton Inference Server. <https://developer.nvidia.com/triton-inference-server/>.
- [45] Open AI. <https://openai.com/blog/ai-and-compute/>.
- [46] G. Pan, V. Chodnekar, A. Roy, and H. Wang, "A cost-benefit analysis of on-premise large language model deployment: Breaking even with commercial llm services," *arXiv preprint arXiv:2509.18101*, 2025.
- [47] T.-D. Phan, G. Pallez, S. Ibrahim, and P. Raghavan, "A new framework for evaluating straggler detection mechanisms in mapreduce," *ACM Trans. on Modeling and Performance Evaluation of Computing Systems*, vol. 4, no. 3, pp. 1–23, 2019.
- [48] Pytorch. <https://pytorch.org/>.
- [49] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the society for industrial and applied mathematics*, vol. 8, no. 2, pp. 300–304, 1960.
- [50] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. of USENIX ATC*, 2020.
- [51] P. Sharma, S. Lee, T. Guo, D. Irwin, and P. Shenoy, "Spotcheck: Designing a derivative iaas cloud on the spot market," in *Proc. of EuroSys*, 2015.
- [52] Z. Shen, Y. Cai, K. Cheng, P. P. Lee, X. Li, Y. Hu, and J. Shu, "A survey of the past, present, and future of erasure coding for storage systems," *Trans. on Storage*, vol. 21, no. 1, pp. 1–39, 2025.
- [53] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [54] M. Soleymani, R. E. Ali, H. MahdaviFar, and A. S. Avestimehr, "ApproxIFER: A model-agnostic approach to resilient and robust prediction serving systems," in *Proc. of AAAI*, 2022.
- [55] J. Thorpe, P. Zhao, J. Eyolfson, Y. Qiao, Z. Jia, M. Zhang, R. Netravali, and G. H. Xu, "Bamboo: Making preemptible instances resilient for affordable training of large DNNs," in *Proc. of USENIX NSDI*, 2023.
- [56] Total cost of LLM by Ptolemy. <https://www.ptolemy.com/post/llm-total-cost-of-ownership>.
- [57] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International conference on machine learning*. PMLR, 2021, pp. 10347–10357.
- [58] J. Varia, "Best practices in architecting cloud applications in the aws cloud," *Cloud Computing: Principles and Paradigms*, pp. 457–490, 2011.
- [59] L. Wang, Y. Hu, Y. Liu, R. Xiao, and D. Feng, "Asymmetric coded distributed computation for resilient prediction serving systems," in *Proc. of Euro-Par*. Springer, 2024.
- [60] S. Wang, J. Liu, and N. Shroff, "Coded sparse matrix multiplication," in *Proc. of ICML*. PMLR, 2018, pp. 5152–5160.
- [61] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [62] Z. Wu, W.-L. Chiang, Z. Mao, Z. Yang, E. Friedman, S. Shenker, and I. Stoica, "Can't be late: optimizing spot instance savings under deadlines," in *Proc. of USENIX NSDI*, 2024.

- [63] Q. Yu, M. Maddah-Ali, and S. Avestimehr, "Polynomial codes: an optimal design for high-dimensional coded matrix multiplication," *Proc. of NeurIPS*, vol. 30, 2017.
- [64] C. Zhang, M. Yu, W. Wang, and F. Yan, "MArk: Exploiting cloud services for Cost-Effective, SLO-Aware machine learning inference serving," in *Proc. of USENIX ATC*, 2019.
- [65] B. Zhuang, J. Qiao, M. Liu, M. Yu, P. Hong, R. Li, X. Song, X. Xu, X. Chen, Y. Ma *et al.*, "Beyond benchmarks: The economics of ai inference," *arXiv preprint arXiv:2510.26136*, 2025.